

少看反而答得更好但功劳不在图上 Schema-Agnostic Graph Reasoning Agent for Hybrid Knowledge Graphs

本篇范围声明：这是一篇 12 页的公司白皮书（arXiv 2608.15834v1，2026-08-16 提交，作者来自法国工业软件公司 Oplit 的 R&D 部门）。下面的「全文翻译」按章节推进，覆盖正文第 1 - 9 节的主体（从代码 agent 到图 agent、相关工作、三个 agent 一个基底、UFK-M 基准、实验设置、结果、进一步分析、工业应用的两个完整例子、结论），四张结果表逐格给出；参考文献列表不逐条翻译。

核心内容

Q：这篇的出发点是什么？ A：一个类比：调工具的 agent 在从没见过的代码仓库里也能干活，靠的是一小把通用原语——列目录、读文件、搜字符串。这些工具对项目本身一无所知，是 agent 靠导航把它重建出来的。

而一张带标注的属性图接受完全相同的接口：**列出一个节点的邻居、读一个节点的内容、搜索节点描述，在结构上是同一批操作，只是基底不同。** 所以代码 agent 的导航能力应该能低成本迁移到知识图谱上。

Q：什么叫「混合知识图谱」？ A：作者假设的基底契约很小：每个节点带标识符、自然语言描述、标签和可选属性；关系是有向的主谓宾三元组。而有些节点是「数据表」，背后是一张真实的 DuckDB 表，可以用 SQL 查。

这就是「混合」的意思：文本知识和关系型知识共存在同一张图里，回答一个问题可能需要其中之一，也可能两者都要。

Q：他们在比什么？ A：喂这种基底有两种哲学在竞争。第一种把一切（文档和 schema）序列化进提示词，然后要一个答案。第二种给模型一个有界的上下文和一套工具，让它自己去取需要的东西。这篇论文测量这个取舍，并且设计了对照来把结果归因到原因上。

三个系统跑在同一个基底上：

系统	是什么	工具	轮次预算
GRA	图推理 agent，用七个 unix 风格通用工具导航混合图	ls cat grep sems query think answer	45 次 LLM 调用

系统	是什么	工具	轮次预算
RSA	GRA 把图去掉：在扁平文本文档的分块和表 schema 上做检索	search_text list_tables describe_table + 同样的 query think answer	45 次
SQA	全上下文：提示词里塞完整的图文本描述和完全渲染的 schema，约 17k token	无检索、无导航	最多 6 轮

公平性是靠构造保证的：GRA 和 RSA 跑完全相同的执行循环、逐字共享策略提示块，只在基底和工具集上不同。

七个工具的分工：

工具	作用
ls	列出节点、表、边，在图里定位
cat	完整读一个节点；对表则显示列、键、join 关系和 3 行样例
grep	在 id、文本、属性、边、列上做字面搜索
sems	语义搜索（稠密 + BM25，取前 10）
query	只读 SQL（SELECT/WITH），最多 50 行
think	规划与检查的草稿纸，无副作用
answer	提交最终答案，可带引用和置信度

ls cat grep 这三个名字是刻意取的——它们对应的就是代码 agent 在陌生仓库里用的那三个原语。

Q：基准是什么？ A：UFK-M（Unified Factory Knowledge Model），一个虚构的自行车装配工厂，受真实客户工厂启发但完全合成。一份「奠基文本」用散文陈述它的运营规则、KPI 和行业概念；然后两个协调的层把这个世界变成机器可读的——数据层（DuckDB 表）和语义层（知识图谱，把奠基文本蒸馏出来并映射到表上）。

层级	表数	行数	图节点	图边	奠基文本词数	问题数
large	32	157,953	125	268	4,278	148
xlarge（主评测）	64	174,006	235	513	6,933	258

Q：问题是怎么生成的？这一步是这篇最值得抄的东西。 A：答案先行的问题生成（Answer-first question generation）——问题是反着造的，答案在问题存在之前就已经定下来了。

流程是：对每个问题，让一个 LLM 拿到一批 schema 卡片（表和列的描述），先写一段 SQL 程序。然后把这段程序在数据库上真跑一遍。只有当结果非空、非退化、且不超过十行时才保留。只有到这一步之后，才让 LLM 去写一个「被这个保留下来的结果所回答」的自然语言问题。

这个顺序很关键（作者原话）：因为 SQL 是在问题之前被写出来并验证过的，所以集合里的每一个问题都保证能从数据里回答出来，而它的标准答案是一个真正跑过的程序的输出，而不是模型产出的文本。

冻结的 xlarge 集有 258 个这样的问题：116 个表答案、84 个单值、48 个布尔、10 个列表；147 个最多需要一次 join、45 个需要两次、66 个需要三次或更多；34 个额外需要语义层（一条具名运营规则或 KPI，其解析值或公式是藏起来的）。

Q：怎么判对错？ A：确定性匹配器，不是 LLM 裁判。数值答案按舍入精度带容差比较；百分比答案按无标度方式比较（不管表达成分数还是百分数）；表答案按「必须包含的标准行的召回率」打分。

Q：结果如何？ A：xlarge 集（n=258），95% 配对自举区间：

模型	GRA	RSA	SQA	ρ （GRA 相对 SQA 的相对错误减少）
DeepSeek V4-Pro-Think	88.4±3.9	87.4±3.9	83.3±4.5	+30.5%
DeepSeek V4-Flash	87.6±3.9	86.1±4.2	84.9±4.5	+17.9%
DeepSeek V4-Pro	87.2±4.1	86.9±3.9	82.6±4.5	+26.4%
GLM-4.5-Air	77.5±5.1	75.6±5.2	74.0±5.3	+13.5%
Qwen3-Coder-Flash	78.7±4.9	78.3±5.0	81.0±4.9	-12.1%
GPT-5 Nano	70.5±5.9	70.2±5.5	76.0±5.3	-22.9%
GPT-5 Nano（开思考）	76.7±5.1	—	—	—

排名会随模型翻转：GRA 在 DeepSeek 和 GLM 上最好，而 SQA 在 Qwen3-Coder-Flash（+2.3pp）和 GPT-5 Nano（+5.5pp）上最好。作者的结论是：当工具使用不可靠时，全上下文推理更稳健。

Q：读了多少 token？ A：GRA 读 SQA 唯一输入 token 的 29 - 33%，RSA 读 24 - 29%。图导航比扁平文本检索读得稍多，但两个 agent 都远低于全上下文基线。

输出 token 的模式反过来：SQA 每问产生 0.5 - 1.3k 补全 token（因为它用单个 SQL 块作答），而多轮 agent 产生 1.0 - 2.0k。

作者对计费的说明很实在：唯一 token 和计费 token 不是一回事。GRA 和 RSA 在 11 - 15 轮里反复重发一个不断增长的上下文，而 SQA 平均不到三轮。在缓存感知的定价下，相对成本取决于工作负载——热缓存的批量评测偏向 SQA（它那个大的固定前缀能跨问题复用），冷启动的单问服务偏向 agent（SQA 每个问题都要吃它完整的 17k token 提示词）。

背景补充 / 点评

这篇最重要的发现是一个反直觉的对照结果

标题里那句「少看反而答得更好」是真的，但功劳不在图上。

作者专门设了 RSA 这个去掉图的对照组，结果是：

GRA 只比 RSA 高 +0.3 到 +1.9pp，而且在 DeepSeek V4-Pro 上两者不可区分，说明相对 SQA 的增益主要来自上下文被如何访问，而不是来自图结构本身。

这一段是全文最有价值的地方。如果没有 RSA 这个对照，读者会自然地把 GRA 的 +5.1pp 归因于「知识图谱有用」。加了对照之后才知道：真正起作用的是「按需取用」这个访问模式，图只贡献了不到 2pp，而且置信区间重叠。

作者在摘要里就把这一点说出来了：「A graph-free control shows the gain comes chiefly from selective agentic access rather than graph topology」。在自己的白皮书里主动削弱自己产品名字里那个「Graph」的分量，这不常见。

工具可靠性比推理深度更要紧

这是第二个有分量的发现，而且有一张干净的表支撑：

模型（GRA）	准确率	平均工具调用	失败调用占比	至少一次失败的问题占比
DeepSeek V4-Pro-Think	88.4%	12.1	0.5%	5.4%
DeepSeek V4-Flash	87.6%	14.8	0.7%	8.1%
DeepSeek V4-Pro	87.2%	13.4	0.6%	7.0%
Qwen3-Coder-Flash	78.7%	12.2	2.0%	15.9%
GLM-4.5-Air	77.5%	11.1	2.4%	18.2%
GPT-5 Nano（开思考）	76.7%	11.6	5.8%	34.9%
GPT-5 Nano	70.5%	11.1	10.2%	51.6%

准确率和失败率严格同序。三个 DeepSeek 配置失败率都低于 1%、占据前三名且相对 SQA 增益最大；GPT-5 Nano 失败率 10.2%，是最不准的配置，也是 GRA 输给 SQA 最多的那个。

开推理对 DeepSeek 几乎没用：三个配置彼此相差在 1.2pp 以内、区间重叠。GPT-5 Nano 是唯一例外（+6.2pp），但它的失败率同时大约减半（10.2%→5.8%，51.6%→34.9%），所以这个增益看起来是通过可靠性而不是推理深度中介的。

这条对我最有用：选模型跑 agent 时，要看的第一个指标是工具调用失败率，不是推理能力。而且这个指标可以直接测。

工具调用预算有一个膝点

预算 B 在 {10, 20, 30, 40, 50} 上扫：准确率从 10 到 30 急剧上升，然后走平。

- xlarge 上 B=10 时准确率只有 63.6%
- 同一区间里，被截断的问题数从 118 降到 11（large 上从 62 降到 3）

- 截断变罕见之后，平均轮次稳定在 large 上约 11、xlarge 上约 13
- 30 次以上观察不到可测增益

所以 $B \approx 30$ 是这些数据集的够用值。而实际用掉的只有 11 - 13 次——预算的作用是给搜索留余量，不是让它真去用满。

两个工业应用的例子，说明了这套东西真正的位置

第 8 节把 GRA 放进一个更大的循环里：操作员用大白话说一条规则 → 编排器接到后问 GRA 这条规则是否可行 → GRA 用那七个通用工具导航图、收集证据、返回一个带引用的判定 → 通过后交给 ORA（运筹 agent）编译成数学优化模型和求解器代码，再注册回图里成为新的规则节点。

第二个例子值得完整看一遍，因为它展示了「答案不在任何地方存着」这件事：

操作员说「1 号线每班次最多三次换色」。GRA 分四步：

1. 找出这些词指向什么：「换色」经一个概念节点指向记录每次换型的表；「班次」指向工作日历（规则 R11：每天两班、每班八小时）
2. 查冲突：列出已经管着 1 号线的规则，没有和这条请求矛盾的
3. 重放历史：没有任何存储的值能判定这条规则。所以 agent 需要去数记录在案的 428 个班次里每一班应到的颜色数
4. 量化风险：其中 11 个班次不可避免地超过三次换色。在那些日子里，这条规则会让排产变得不可能

判定：可行，但有可测的季节性风险。agent 给出两种采纳方式——作为硬上限并对高峰日留人工覆盖，或作为软约束在目标函数里罚分。操作员选了硬形式。

作者点出的那句是关键：「和路径 1 一样，这个判定作为存储的文本在任何地方都不存在；它是在提问的那一刻从表里算出来的。」

部署时只加了一个写原语 `edit`，只用来注册被批准的规则，不在被评测的工具集里。而且 ORA 产出的数学表述和确切代码行是作为 pull request 发给 Oplit 的运筹专家评审批准后才合并的——人在环里，写回是受控的。

三处必须打折扣的地方，其中两处作者自己说了

第一，基准是合成的。UFG-M 是一个虚构的自行车装配工厂，「受真实客户工厂启发但完全合成」。作者说了。合成基准的好处是标准答案干净（SQL 真跑过），代价是不知道真实工厂的数据脏乱程度会怎样影响结果。

第二，最该起作用的那个场景没测。作者在第 7 节末尾自己写了：

当前语料小到 SQA 的 17k token 提示词能舒服地装进每个模型的上下文窗口，所以「结构化导航本

应最有帮助」的那个区制——序列化不可行或成本过高的时候——仍有待在实质上更大的图上检验。

这一条几乎抵消了论文的主要卖点。「按需取用比全塞进去好」这个结论，是在「全塞进去完全塞得下」的条件下测出来的。真正需要这套方法的场景，恰恰是还没测的那个。

第三，这一条作者没说：结论对模型的依赖强到影响可推广性。六个模型里，两个模型上 SQA 反而更好（Qwen3-Coder-Flash +2.3pp、GPT-5 Nano +5.5pp）。作者把这归因于工具调用可靠性，这个归因有表 4 支撑、我认为成立。但它意味着「用 agent 还是全上下文」不是一个可以一次性决定的架构选择，而是一个依赖于你用哪个模型的运行时选择——而模型是会换的。论文没有讨论这件事对工程决策的影响。

我的判断

这篇的方法论价值高于它的结论价值。

结论部分（+5.1pp、读三分之一 token）受限于合成基准和「装得下」的语料规模，可推广性有限。但两样东西可以直接搬走：答案先行的问题生成，和去掉一个成分做对照这个实验设计。

尤其是后者。如果没有 RSA，这篇就是又一个「知识图谱提升了 X%」的论文；有了 RSA，它变成了「提升来自访问模式，图只值不到 2pp」。这两个结论指导的工程决策完全不同——前者会让你去建图，后者会让你先去做选择性检索。

对我的启示

第一条，答案先行的问题生成可以直接搬到 prove 的评测集上。

我们现在的评测题硬编码在 Python 字典里（8/31 读 data.world 那个基准时确认的），标准答案是人写的。这篇给了一条更好的路：

给 LLM 一批 schema 卡片 → 让它先写 SQL → 真跑一遍
→ 结果非空、非退化、≤10 行才保留 → 再让 LLM 写自然语言问题

这个顺序保证了两件事：每道题都确实能从数据里答出来（不会出现「这题本来就没答案」）；标准答案是程序输出而不是模型文本。

而且它自动带来一个我们缺的东西：题目的结构化标注。这篇的 258 题自动就分好了——147 题最多一次 join、45 题两次、66 题三次以上；34 题额外需要语义层。这个划分不用人标，是从生成时那段 SQL 的结构里直接读出来的。

「34 题额外需要语义层」这个切分尤其重要，它正是我们缺的那个消融：有多少题真的需要本体，有多少题纯 SQL 就够了。没有这个切分，本体的价值说不清楚。

第二条，确定性匹配器的三条规则可以照抄。

数值按舍入精度带容差；**百分比无标度比较**（不管写成 0.82 还是 82%）；**表答案按标准行的召回打分**（而不是要求完全一致）。

这三条正好对上 8/31 那个 data.world 基准的三条豁免规则（列顺序不同不算错、列名不同不算错、行顺序不同不算错），**但更具体、更可实现**。两边合起来就是一套完整的判分口径。

第三条，「去掉一个成分做对照」这个动作，是我这两天反复吃亏的地方的解药。

我在 roleplay 评估 CSRS 替代方案时，**第一版把 CSRS 的能力清单直接抄成了收益清单**，被崔扬逐条驳回。事后我总结的教训是「要逐条问这个能力我用得上吗」。

这篇给了一个更强的做法：把那个成分去掉，看差多少。 RSA 就是「把图去掉的 GRA」，一跑就知道图值多少。

具体到 prove：如果要论证「本体带来了多少价值」，**正确的对照不是「有本体 vs 没有任何东西」，而是「有本体 vs 有一个同等工程量但没有本体的检索方案」**。前者一定赢，后者才说明问题。

第四条，选跑 agent 的模型时先看工具调用失败率。

表 4 那个严格同序的关系（失败率 0.5% → 88.4%，失败率 10.2% → 70.5%）说明：**在这类任务上，工具可靠性是比推理能力更强的瓶颈**。而且它是可以直接测的——跑一批题，数失败调用占比和「至少失败一次的问题占比」两个数。

这条对我手上的流水线也适用：那些跑脚本、查数据、发 Slack 的例行任务，**失败模式几乎全是工具层面的**（数据源返回过期值、脚本没有只读模式、cron 错时触发），不是推理层面的。

文中金句

"A graph-free control shows the gain comes chiefly from selective agentic access rather than graph topology, and that the effect depends on a model able to drive tools reliably."

一个**去掉图的对照组**表明，增益主要来自**选择性的 agent 式访问**而不是图的拓扑结构，而且这个效应依赖于一个能可靠驱动工具的模型。

"Seeing less, the agent answers better: selective navigation over a structured substrate

beats exhaustive context." **看得更少，agent 答得更好**：在结构化基底上的选择性导航，胜过穷尽式的上下文。

"Because the SQL is written and validated before the question, every question in the set is answerable from the data, and its gold answer is the output of a program that has actually

run rather than text produced by a model." 因为 SQL 是在问题之前被写出并验证过的，集合里每个问题都能从数据中答出，而它的标准答案是一个真正跑过的程序的输出，而不是模型产出的文本。

"These results indicate that reliable tool use is a stronger bottleneck than extended reasoning on this task." 这些结果表明，在这个任务上，可靠的工具使用是比延长推理更强的瓶颈。

"The current corpus is small enough that SQA's 17k-token prompt fits comfortably in every model's context window, so the regime in which structured navigation should help most, where serialization is infeasible or too costly, remains to be tested on substantially larger graphs." 当前语料小到 SQA 的 17k token 提示词能舒服地装进每个模型的上下文窗口，所以「结构化导航本应最有帮助」的那个区制——序列化不可行或成本过高的时候——仍有待在实质上更大的图上检验。

全文翻译

摘要

Tool-calling LLM agents navigate unfamiliar codebases with a handful of generic primitives for listing, reading and searching files (`ls`, `cat`, `grep`). A knowledge graph admits the same interface: listing neighbours, reading node content and searching descriptions are the same operations on a different substrate.

调用工具的大模型 agent，靠一小把用于列出、读取和搜索文件的通用原语（`ls`、`cat`、`grep`）在陌生代码库里导航。知识图谱接受相同的接口：列出邻居、读取节点内容、搜索描述，是同一批操作作用在不同的基底上。

Building on this correspondence, we present GRA, a Graph Reasoning Agent that explores hybrid knowledge graphs, whose nodes are either textual concepts or relational tables, with seven generic tools, discovering everything domain-specific at run time.

基于这个对应关系，我们提出 GRA（图推理 agent），它用七个通用工具探索混合知识图谱——图的节点要么是文本概念、要么是关系表——在运行时发现一切领域专属的东西。

On UFK-M (Unified Factory Knowledge Model), an industrial benchmark of 258 analytical questions whose gold answers are produced by executing validated SQL programs, GRA beats a full-context agent by 5.1 pp (88.4% vs. 83.3%), while reading under a third of its input tokens.

在 UFK-M（统一工厂知识模型）上——一个含 258 道分析题的工业基准，其标准答案由执行经过验证的 SQL 程序产生——GRA 比全上下文 agent 高 5.1 个百分点（88.4% 对 83.3%），同时读取不到它三分之一

的输入 token。

A graph-free control shows the gain comes chiefly from selective agentic access rather than graph topology, and that the effect depends on a model able to drive tools reliably. Seeing less, the agent answers better: selective navigation over a structured substrate beats exhaustive context.

一个去掉图的对照组表明，增益主要来自选择性的 agent 式访问而不是图拓扑，且该效应依赖于一个能可靠驱动工具的模型。看得更少，agent 答得更好：在结构化基底上的选择性导航胜过穷尽式上下文。

一、从代码 agent 到图 agent

Tool-calling agents work effectively inside repositories they have never seen, using a small set of generic primitives: list a directory, read a file, search for a string. None of these tools knows anything about the project; the agent reconstructs it by navigating.

调用工具的 agent 在它们从未见过的仓库里也能有效工作，用的是一小组通用原语：列目录、读文件、搜字符串。这些工具没有一个知道关于这个项目的任何事；是 agent 靠导航把它重建出来的。

一张带标注的属性图接受相同的接口。列出一个节点的邻居、读取一个节点的内容、搜索节点描述，在结构上是作用于不同基底的等价操作，所以代码 agent 的导航能力应该能以很小的代价迁移到知识图谱上。

我们假设的基底契约是最小的。每个节点带一个标识符、一段自然语言描述、若干标签和可选属性；关系是有向的主语—谓词—宾语三元组。有些节点是数据表，背后是一张真实的、可用 SQL 查询的 DuckDB 表。这就是「混合知识图谱」里的「混合」：文本知识和关系型知识共存于一张图中，回答一个问题可能需要其中之一，也可能两者都需要。

Two philosophies compete to feed such a substrate. The first serializes everything (documentation and schema) into the prompt and asks for an answer. The second gives the model a bounded context and a toolset, and lets it fetch what it needs. This paper measures that trade-off, with controls designed to attribute the outcome to its cause.

有两种哲学在竞争如何喂养这样的基底。第一种把一切（文档和 schema）序列化进提示词然后要一个答案。第二种给模型一个有界的上下文和一套工具，让它自己去取需要的东西。本文测量这个取舍，并配以旨在把结果归因到其原因的对照。

二、相关工作

GRA 的接口承自 ReAct，后者引入了推理与工具调用交错这一如今在大模型 agent 中常见的做法，但留下了「某个给定基底应当暴露哪些工具」这个开放问题。对于源代码，SWE-agent 回答了这个问题：它的 agent - 计算机接口表明，少数几个文件系统命令就足以在一个陌生仓库上工作。知识图谱在这一点上很像代码库——一个通过跟随链接而不是整体通读来探索的大型连通结构——所以策略可以直接迁移，只是基底不同。

另一条平行的线直接给语言模型一个通向结构化知识的接口，如图检索增强生成的综述所述。这些系统在图、表和数据库上读取，跟随关系路径，或暴露节点查找和邻居列举的原语。**它们都共享 GRA 的前提——导航图而不是读它的扁平转储——但假设图的词汇表是已知的，且仅靠遍历就能回答问题。**

Neither assumption holds on a hybrid substrate, where the deciding fact often sits in a table reached through the graph, and where a question's words must first be matched to a node.

在混合基底上这两个假设都不成立：决定性的事实常常坐落在一张要经由图才能到达的表里，而一个问题的词必须先被匹配到某个节点上；对齐这两者本身就很难，自然语言到图查询翻译的工作已经表明了这一点。

GraphRAG 则是离线构建图和摘要再在其上检索，这适合宽泛的问题但把检索结构预先固定了，而 GRA 是在一张持续变化的图上逐问题地收集证据。

Each of these provides part of what GRA needs, but none provides all of it at once: operation without per-schema tuning, a substrate that unites a semantic graph with relational tables, and the ability to compute a quantity no node stores.

上述每一项都提供了 GRA 所需的一部分，但没有一项同时提供全部三样：无需逐 schema 调优的运行、把语义图与关系表统一起来的基底、以及计算一个没有任何节点存储的量的能力。

三、三个 agent，一个基底

3.1 GRA（七个工具见「核心内容」一节）

GRA explores the graph with the seven unix-style tools of Table 1. Nothing domain-specific is hard-coded in the tools or the system prompt: no concept list, no label strings, no table names.

GRA 用表 1 那七个 unix 风格的工具探索图。**工具和系统提示词里没有硬编码任何领域专属的东西：没有概念清单、没有标签字符串、没有表名。**

一个 GRA 在 UFK-M 上的例子。问题：「上季度每个客户细分市场有多少订单延迟了？」agent 是冷启动的：没有 schema、没有表名、没有词汇表。

轨迹：`ls(labels)` → `ls(tables)` → `grep("orders")` → `grep("customer segment")` → `cat(tbl_customer_orders)` → `cat(Customer)` → `cat(tbl_customers)` → `query(...)` → `think` → `answer`。

结果：知识图谱和「延迟」的定义是在路上被发现的，而全程只读了几千个唯一 token。

3.2 RSA：检索式 SQL agent

RSA is GRA with the graph removed: it retrieves over chunks of the flat textual documentation and over table schemas, and keeps the identical query, think and answer tools. Fairness is enforced by construction: both agents run the very same execution loop, share their strategy prompt blocks verbatim, and differ only in substrate and toolset.

RSA 就是把图去掉的 GRA：它在扁平文本文档的分块和表 schema 上做检索（`search_text`、`list_tables`、`describe_table`），并保留完全相同的 `query`、`think`、`answer` 工具。公平性由构造强制保证：两个 agent 跑完全相同的执行循环，逐字共享它们的策略提示块，只在基底和工具集上不同。

同一个问题上，RSA 的轨迹是：`search_text("customer segment")` → `list_tables()` → `describe_table(tbl_customer_orders)` → `describe_table(tbl_customers)` → `query(...)` → `think` → `answer`。

结果：同一个循环到达同一个答案，但「细分市场这个概念」和「存储它的那张表」之间的联系，必须从检索到的文本块里推断出来，而不是从一条边上读出来；候选表是按名字扫的，不是被跟随的。

3.3 SQA: SQL agent

SQA 走「序列化一切」的路线：它收到一个包含图的完整文本描述和完全渲染的 schema（带类型的列、键关系、类别词汇表、日期范围）的提示词，**总计约 17k token**，并在**最多六轮**内产出 SQL 答案。

结果：没有搜索、没有导航，`join` 路径从一开始就在提示词里可见。整个语料在第一个词之前就被读完了，无论这个问题需不需要，而剩下的轮次只用来修复失败的查询。

四、UFG-M 基准

UFG-M 是一个虚构的自行车装配工厂，**受真实客户工厂启发但完全合成**。一份奠基文本用散文陈述它的运营规则、KPI 和行业概念；随后两个协调的层把这个世界变成机器可读的：**数据层**（DuckDB 表）和**语义层**（知识图谱，把奠基文本蒸馏出来并映射到表上）。两个嵌套层级在不移除信息的前提下缩放基准，本文主要在 xlarge 上评测。

答案先行的问题生成。问题是反着生成的：**答案在问题存在之前就已固定**。对每个问题，一个 LLM 收到一批 schema 卡片（表和列的描述）并在其上写一段 SQL 程序。**该程序随后在数据库上被执行**。只有当它的结果非空、非退化、且至多十行时才被保留。只有到那时，LLM 才写出一个「被保留下来的那个结果所回答」的自然语言问题。

This order matters. Because the SQL is written and validated before the question, every question in the set is answerable from the data, and its gold answer is the output of a program that has actually run rather than text produced by a model.

这个顺序很重要。因为 SQL 是在问题之前被写出并验证过的，集合里的每个问题都能从数据中答出，而它的**标准答案是一个真正跑过的程序的输出**，而不是一个模型产出的文本。

冻结的 xlarge 集有 258 道这样的题：**116 个表答案、84 个单值、48 个布尔、10 个列表**；**147 道最多需要一次 join、45 道需要两次、66 道需要三次或更多**；**34 道额外需要语义层**（一条具名的运营规则或 KPI，其解析值或公式是隐藏的）。

确定性评分。正确性由**确定性匹配器判定而不是 LLM 裁判**。数值答案按舍入精度带容差比较；百分比答案按无标度方式比较（与它被表达成分数还是百分数无关）；表答案按「它必须包含的标准行的召回率」打分。

五、实验设置

评测七个骨干配置、跨四个供应商：DeepSeek V4-Flash（非思考，参考基准）、DeepSeek V4-Pro 与 V4-Pro-Think（同一份权重，推理关闭与开启）、GPT-5 Nano（低/高推理强度）、GLM-4.5-Air、Qwen3-Coder-Flash。

每个骨干在三个 agent 基线下运行，GRA/RSA 给 45 次 LLM 调用轮次，SQA 给 6 次；SQL 工具结果上限 50 行，GRA/RSA 通过本地嵌入器 multilingual-e5-large-instruct 检索，SQA 则在提示词里收到完整 schema、不做检索。所有模型贪心解码（temperature 0），补全上限 1024 token（思考配置为 8192）以避免截断答案；其余部分的执行框架在各系统间完全相同。不确定性通过在问题上做配对自举量化（ $B = 10^4$ ），给出 95% 分位区间。

六、结果

6.1 各模型上的准确率（表见「核心内容」一节）

DeepSeek 系模型在所有系统上都取得最高准确率，在 GRA 和 RSA 下高出 8 - 18pp，但在 SQA 下只高出 2 - 9pp。在低成本模型中，Qwen3-Coder-Flash 在每个系统上都显著优于 GPT-5 Nano。

The ranking of agentic and full-context methods depends on the model: GRA performs best with the DeepSeek and GLM models, whereas SQA performs best with Qwen3-Coder-Flash (+2.3pp over GRA) and GPT-5 Nano (+5.5pp). Full-context inference is therefore more robust when tool use is unreliable.

agent 式方法与全上下文方法的排名取决于模型：GRA 在 DeepSeek 和 GLM 模型上表现最好，而 SQA 在 Qwen3-Coder-Flash（比 GRA 高 2.3pp）和 GPT-5 Nano（高 5.5pp）上表现最好。因此当工具使用不可靠时，全上下文推理更稳健。

6.2 工具可靠性比延长推理更要紧（表见「背景补充」一节）

延长推理在工具使用已经可靠的地方几乎没有改变：三个 DeepSeek 配置彼此相差在 1.2pp 以内，区间重叠。GPT-5 Nano 是例外，为 +6.2pp，但它的失败率同时大约减半（失败调用 10.2%→5.8%，有失败的问题 51.6%→34.9%），所以这个增益看起来是由可靠性而非推理深度中介的。

准确率在整个网格上都跟随这个度量：DeepSeek 各配置失败率低于 1% 并占据前三名、相对 SQA 增益最大；GPT-5 Nano 失败 10.2%，既是最不准确的配置，也是 GRA 输给 SQA 最多的那一个。

6.3 Token 用量

图 1 报告的是**唯一输入 token**，每个 token 在一条轨迹里只计一次。这测量的是语料覆盖度而不是计费用量。五个模型上，GRA 读 SQA 唯一输入的 29 - 33%，RSA 读 24 - 29%。因此图导航比扁平文本检索读得略多，但两个 agent 都远低于全上下文基线。

输出 token 的模式相反：SQA 每问产生 0.5 - 1.3k 补全 token，因为它以单个 SQL 块作答；而多轮 agent 在迭代搜索和规划中产生 1.0 - 2.0k。

Unique and billed input tokens differ. GRA and RSA agents resend a growing context over 11 - 15 turns, whereas SQA averages fewer than three turns. Under cache-aware pricing, the relative cost depends on the workload. Warm-cache batch evaluation favours SQA because its large fixed prefix can be reused across questions. Cold-start single-question serving favours GRA and RSA agents because SQA incurs its full 17k-token prompt on every question.

唯一 token 和计费 token 是不同的。 GRA 和 RSA 在 11 - 15 轮里反复重发一个不断增长的上下文，而 SQA 平均不到三轮。在缓存感知的定价下，相对成本取决于工作负载。热缓存的批量评测偏向 SQA，因为它那个大的固定前缀可以跨问题复用。冷启动的单个服务偏向 GRA 和 RSA，因为 SQA 每个问题都要付它完整的 17k token 提示词。

6.4 工具调用预算的影响

每个问题受一个固定的工具调用预算约束。**如果预算在找到相关数据之前耗尽，agent 必须在完成搜索之前作答。** 预算 B 在 {10, 20, 30, 40, 50} 上变化，用的是**预算感知**的 GRA（它在开始时收到预算，并在消耗约 80% 后收到警告）。除准确率外，还测量**截断率**（耗尽预算而没有自愿 `answer` 的问题占比）和平均工具调用轮次。

准确率在 10 到 30 之间急剧上升，然后走平。同一区间内，**被截断的问题数在 large 上从 62 降到 3、在 xlarge 上从 118 降到 11**。一旦截断变得罕见，平均轮次稳定在 large 上约 11、xlarge 上约 13，**额外预算几乎没有影响**。低于 20 次的预算频繁截断搜索并降低准确率；xlarge 上 B=10 时准确率为 63.6%。30 次以上观察不到可测增益，**因此约 30 次调用的预算对这些数据集是足够的。**

七、进一步分析

Taken together, the results identify selective agentic access as the primary source of GRA's advantage on this benchmark.

综合来看，这些结果指认**选择性的 agent 式访问**是 GRA 在这个基准上优势的**首要来源**。在能可靠调用工具的模型上，两个 agent 都优于 SQA，而 GRA 取得最大的、有良好支撑的增益 **+5.1pp**，**同时只读取 SQA 唯一输入 token 的 29 - 33%**。

The contribution of graph topology, however, is much less clear: GRA exceeds RSA by only +0.3 to +1.9 pp, and the two are indistinguishable on DeepSeek V4-Pro, indicating that the gain over SQA comes mainly from how context is accessed rather than from graph structure itself.

然而图拓扑的贡献要不清楚得多：GRA 只比 RSA 高 +0.3 到 +1.9pp，且两者在 DeepSeek V4-Pro 上不可区分，说明相对 SQA 的增益主要来自上下文被如何访问，而不是来自图结构本身。

可靠性设定了这一行为的边界：一旦工具调用失败超过百分之几，agent 式框架的优势就缩小甚至反转，这就是为什么全上下文推理对较弱的工具调用者仍然更可取。

最后，当前语料小到 SQA 的 17k token 提示词能舒服地装进每个模型的上下文窗口，所以「结构化导航本应最有帮助」的那个区制——序列化不可行或成本过高时——仍有待在实质上更大的图上检验。

八、GRA 带来工业智能——完整概览

前面几节把 GRA 当作一个问答模块来测量。在一家工厂里，同一个模块坐落在一个更大的循环中：操作员用大白话陈述一条规则；编排器接到它并问 GRA 这条规则是否可行；GRA 用表 1 那些通用工具导航图来收集证据，返回一个带引用的判定。

部署时增加了一个写原语 `edit`，只用于注册被批准的规则；它不在被评测的工具集里。被接受的规则交给 ORA（运筹 agent），后者把它们转成数学优化模型和求解器代码，并作为新的规则节点注册回图中。

8.2 例二 —— 把一条被接受的规则编译进排产模型（完整走查见「背景补充」一节）

人工批准与写回。数学表述和确切的代码行作为一个 pull request 发给 Oplit 的运筹专家，由他们评审并批准之后才会被合并。

九、结论

论文的结论重申三点：选择性访问优于穷尽上下文（在能可靠调用工具的模型上）；这个增益主要来自访问模式而非图拓扑（去图对照组证实）；以及这套方法真正的用武之地——序列化不可行的大规模语料——尚待检验。

📌 参考链接

- [arXiv:2608.15834](#) — 本文原文（cs.AI, 2026-08-16 提交）
- [Oplit](#) — 作者所属公司，做工业排产与运筹优化
- [ReAct \(Yao 等\)](#) — GRA 接口的来源，推理与工具调用交错
- [SWE-agent](#) — 「agent - 计算机接口」，证明少数文件系统命令足以在陌生仓库工作
- [GraphRAG \(微软\)](#) — 离线建图加摘要再检索，本文对照的另一条路线
- [DuckDB](#) — UFK-M 数据层所用的嵌入式分析数据库
- [multilingual-e5-large-instruct](#) — GRA/RSA 检索所用的本地嵌入器
- [MiniZinc](#) — ORA 编译目标，例二里的 `scheduling.mzn` 求解器代码